

Sección 4 — Juegos

Documentación técnica: Codenames determinista por semilla, escalera de palabras (grafo de Hamming 1 + BFS) y crucigrama (colocación por backtracking)

1. Qué resuelve esta sección

La Sección 4 convierte el vocabulario de la plataforma en **juegos de palabras**, cada uno con un algoritmo clásico como corazón: el **Codenames** (Indescifrable) reparte un tablero 5×5 con un generador congruencial lineal, de modo que una semilla de 6 caracteres reemplaza a un servidor; la **escalera de palabras** pide transformar una palabra alemana en otra cambiando una letra por paso — un camino en el grafo de Hamming 1 que se resuelve con BFS; y el **crucigrama** coloca palabras alemanas entrelazadas con **backtracking**, con las pistas en español. Los tres comparten el principio de la plataforma: **sin APIs en vivo** — todo es determinista por semilla, así que compartir una partida es compartir un string.

La escalera y el crucigrama son además **pedagógicos**: sus palabras salen del corpus de lecturas (Sección 1), cada peldaño de la escalera muestra su traducción y las pistas del crucigrama son las glosas españolas del léxico. Jugar es repasar vocabulario.

2. Mapa de archivos

Pieza	Archivo	Qué contiene
Selección de palabras	pipeline/juegos.py	Extrae de lexico.json las formas ASCII (L=3–5) con glosa para la escalera y 400 lemas frecuentes no funcionales como entradas del crucigrama
Datos	src/data/juegos.json	escalera{L: {palabra: glosa}} y crucigrama[{palabra, pista}]; lo sobrescribe el pipeline
Motor escalera	src/engine/escalera.js (+ .test)	Grafo por cubetas comodín, BFS (camino mínimo y niveles), generación de retos por semilla, validación de pasos, estadísticas
Motor crucigrama	src/engine/crucigrama.js (+ .test)	Reglas de colocación, búsqueda por backtracking con presupuesto, numeración clásica, cuadrícula y métricas
Motor Codenames	src/engine/board.js (+ .test)	LCG por semilla, Fisher–Yates, composición/parseo de semilla con código de vocabulario
UI	src/secciones/juegos/*.jsx + juegos.css	/juegos (hub), /juegos/escalera, /juegos/crucigrama; Codenames en /juegos/codenames (src/JuegoApp.jsx, previo a esta sección)
Estadísticas	simulacion/juegos-stats.mjs	Ejecuta los motores reales → docs/datos-juegos.json (lo consume el PDF de métricas)

3. Datos: pipeline/juegos.py → juegos.json

El pipeline no inventa vocabulario: filtra el léxico de la Sección 1. Para la **escalera** toma las formas alemanas de 3–5 letras **solo ASCII** (sin umlauts ni ß: el alumno hispanohablante debe poder teclearlas) con su

traducción. Para el **crucigrama** toma **lemas** de 4–9 letras atestiguados ≥ 2 veces en las lecturas (frecuencias de la Sección 2), excluyendo una lista curada de **palabras funcionales** (artículos, preposiciones, pronombres, auxiliares): «der» es frecuentísimo pero la pista «el / la» no enseña nada. Quedan las 400 más frecuentes, ordenadas por frecuencia.

```
{ "escalera": { "4": { "haus": "casa", "hand": "mano", ... }, ... },  
  "crucigrama": [ { "palabra": "sagen", "pista": "decir" }, ... ] }
```

Como el resto de datos pesados, `juegos.json` entra al frontend por **dynamic import**: chunk aparte, fuera del bundle inicial.

4. Escalera de palabras: grafo de Hamming 1 + BFS

4.1 El modelo

Nodos = palabras del diccionario de longitud L ; arista entre dos palabras si difieren en **exactamente una letra** (distancia de Hamming 1). Un reto es un par (origen, destino) y jugar es recorrer un camino: cada paso debe ser una palabra del diccionario vecina de la anterior. El «par» perfecto lo certifica el **camino mínimo**, que BFS encuentra en $O(V + E)$ por ser el grafo no ponderado.

4.2 Construcción por cubetas comodín

Comparar todas las parejas es $O(n^2 \cdot L)$. En su lugar, cada palabra se mete en L cubetas, una por posición enmascarada (`hand` $\rightarrow$ `*and`, `h*nd`, `ha*d`, `han*`): dos palabras son vecinas **si y solo si** comparten alguna cubeta. La construcción queda en $O(n \cdot L)$ más el tamaño real de la salida, y palabras de longitudes distintas nunca colisionan (la máscara conserva la longitud).

```
hand  $\rightarrow$  { *and, h*nd, ha*d, han*}  
land  $\rightarrow$  { *and, l*nd, la*d, lan*}    comparten *and  $\rightarrow$  vecinas
```

4.3 Generación de retos

`generarReto(palabras, {pasos, semilla})` baraja los orígenes posibles con el LCG (determinista) y, para cada uno, calcula los niveles BFS y busca destinos a distancia **exacta** «pasos»: el reto anuncia su dificultad y la garantiza (no existe atajo más corto, porque la distancia ES el mínimo). La UI ofrece pista (siguiente palabra de un camino mínimo desde la posición actual, recalculado por BFS), deshacer y rendirse (muestra el camino del reto). Todo peldaño pisado enseña su glosa.

5. Crucigrama: colocación por backtracking

5.1 Reglas de colocación (legalidad)

El tablero es un mapa disperso celda $\rightarrow$ letra. Colocar una palabra en (fila, col, dir) es legal si: (1) la casilla anterior al inicio y la posterior al final están libres; (2) cada casilla de la palabra o está **vacía** o ya contiene **la misma letra** (eso es un cruce); y (3) toda casilla vacía que se rellena no toca lateralmente otra palabra (sin palabras paralelas pegadas). Además, salvo la primera palabra, toda colocación debe tener **al menos un cruce**: el tablero es conexo por construcción.

5.2 La búsqueda

Se parte de la palabra más larga del pool en horizontal (mejor ancla) y se profundiza: para cada palabra restante se generan sus colocaciones legales **ancladas en un cruce** (solo casillas donde ya está una de sus letras: el espacio de búsqueda se reduce a los anclajes posibles), ordenadas de más a menos cruces (tableros densos primero). Si una rama no alcanza el objetivo de n palabras, se **deshace** la colocación (se borran solo las casillas que esa palabra añadió, respetando los cruces) y se prueba la siguiente. Un **presupuesto de nodos** (5.000) acota el peor caso exponencial; si se agota, se reintenta con objetivo $n-1$ — con una palabra nunca falla, así que siempre devuelve tablero. En la práctica el presupuesto no se toca: la tasa de éxito medida es 100% (véase `metricas-seccion4.pdf`).

5.3 Salida

El resultado se traslada a origen (0, 0), se calculan las dimensiones y se numera al estilo clásico: casillas iniciales en orden de lectura; si una horizontal y una vertical empiezan en la misma casilla, comparten número. `cuadrícula()` lo vuelca a una matriz para la UI (`null` = casilla negra) y `metricas()` cuenta casillas, cruces y densidad.

6. Codenames (Indescifrable): la semilla como servidor

El juego original necesita que dos capitanes vean la MISMA clave secreta sin comunicarse. Aquí lo resuelve la aritmética: la semilla (p. ej. DP-K9A2) codifica el vocabulario (DP = alemán principiante) y alimenta un **LCG** ($a = 1103515245$, $c = 12345$, $m = 2^{31}$) que baraja las palabras y reparte los colores con Fisher–Yates. Mismo string → mismo tablero y misma clave en cualquier dispositivo, sin backend. El barajado del resto de juegos reutiliza este LCG con una corrección (`crearGeneradorNormalizado`): el generador original puede emitir valores fuera de $[0, 1)$ con hashes negativos, y la parte fraccionaria lo normaliza sin alterar los tableros ya repartidos.

7. Motor puro (JS) y UI

Los tres motores viven en `src/engine/` sin DOM ni `localStorage`, testeados con Vitest (grafo y BFS sobre diccionarios de juguete y sobre el real; legalidad, determinismo e invariantes del crucigrama: cruces $\geq$ palabras $- 1$, sin conflictos de letra, numeración en orden de lectura). La UI (`src/secciones/juegos/`) solo presenta: **/juegos** es el hub con los tres juegos; **/juegos/escalera** ofrece longitud (3–5) y pasos (3–6), input con validación (¿cambia una letra? ¿existe en el corpus?) y glosa por peldaño; **/juegos/crucigrama** renderiza la cuadrícula interactiva (avance de foco en la dirección activa, click repetido alterna horizontal/vertical, flechas y Backspace), comprobar y revelar. El Codenames conserva su pantalla clásica en `/juegos/codenames`.

8. Decisiones y trampas

Tema	Decisión / trampa
Solo ASCII en escalera/crucigrama	Umlauts y ß se excluyen: en teclado español romperían la fluidez de tecleo. El grafo pierde ~5% de nodos (medido)
Distancia EXACTA en el reto	Buscar destino a distancia exacta k ($no \leq k$) hace que el «mínimo posible» anunciado sea verdad por construcción
Palabras funcionales fuera	Frecuencia alta $\neq$ buena entrada: «der/und/nicht» no enseñan nada como pista; lista curada en <code>pipeline/juegos.py</code>
Formas vs lemas	La escalera usa FORMAS (los peldaños solo deben existir); el crucigrama usa LEMAS (una entrada de crucigrama es una palabra de diccionario)
Presupuesto del backtracking	Garantiza terminación con peor caso exponencial; el reintento $n-1$ garantiza salida. Medido: nunca se activa con el pool real
Imports con extensión .js	Los motores se importan también desde node (<code>simulacion/juegos-stats.mjs</code>): los imports internos llevan extensión explícita

9. Cómo regenerar cada artefacto

```
python pipeline/juegos.py          # src/data/juegos.json (tras añadir lecturas)
npm test                          # tests de los motores
npm run simular-juegos            # docs/datos-juegos.json (estadísticas reales)
python docs/generar_doc_seccion4.py # este PDF
python docs/generar_metricas_seccion4.py
python docs/generar_autoaprendizaje_seccion4.py
```